

THE DELIVERY GAP

Faster coding is only the beginning.

Follow one ordinary change from a vague request to an inspectable decision – then use what it teaches us to improve the next change.

“Let customers cancel their orders.”

The bottleneck moves from producing code to agreeing intent, carrying it through and judging the result.

ROLE 01 / CAPTURE & REFINE

The request is simple. The behaviour isn't.

Before implementation, expose the decisions hidden inside the sentence.

WHO?	The owner, a support agent, or anyone with an order ID?	WHEN?	Before fulfilment starts, while it is in progress, or afterwards?
REPEAT?	Does a repeated request return the same outcome or create new work?	FAILURE?	What remains true if cancellation only partly completes?

The first useful output is not code. It is a decision-ready set of questions.

VERTICAL SLICE / AGREEMENT

Turn the request into an agreement.

A reusable method asks; product-specific knowledge constrains; a human decides.

REQUEST

Cancel an order

Eligibility is unresolved.

QUESTION

After fulfilment starts?

Cancellation or returns?

HUMAN DECISION

Before fulfilment

Afterwards, refuse unchanged;
returns outside scope.

AGREED OUTPUT

Testable behaviour

Eligibility, repeats and failure
states are explicit.

Capture & Refine creates a shared, testable agreement — not merely a longer document.

METHOD × CONTEXT

Separate what travels from what belongs here.

The capability improves when reusable technique and product truth can change independently.

REUSABLE SKILL

How to ask

Permissions · validation · repeats · errors · partial failure

DOMAIN CONTEXT

What applies here

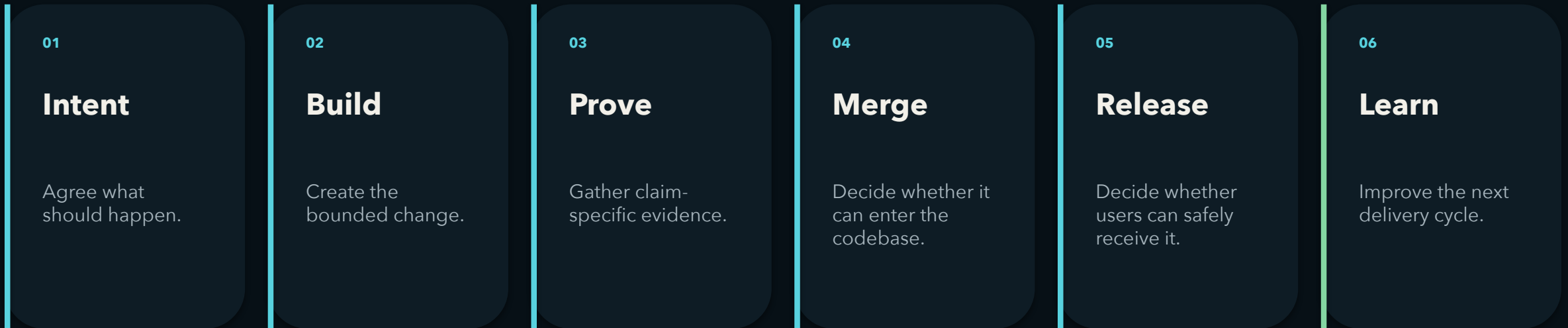
Order states · ownership · fulfilment · payment obligations

The questions can travel across products. The business answers usually cannot.

THE REIMAGINED LIFECYCLE

Carry intent through six explicit decisions.

The seven roles are a practical mapping onto the lifecycle – not a replacement for it.



Intent → Build → Prove → Merge → Release → Learn.

THE FOUR CAPABILITIES OF AN AGENTIC SDLC

Four capabilities make the lifecycle dependable.

They reduce ambiguity, scale merge confidence, automate release confidence and govern decisions with clear authority.

01

Ambiguity reduction

Resolve consequential uncertainty before code.

02

Scalable merge confidence

Inspect current, relevant evidence for a merge decision.

03

Automated release confidence

Assess assembled-system and operational readiness.

04

Engineering governance

Bound authority, approvals, provenance and learning.

Capability connects faster production to confidence and control.

02 / DESIGN

Design makes the behaviour visible before planning the build.

The agreed requirement becomes states, interactions, edge cases and an experience contract.

INPUT

Agreed behaviour

WORK

**States • access •
refusal • recovery**

OUTPUT

Experience contract

STOP

**Unresolved product
decision**

Design precedes planning so implementation is not forced to invent the user experience.

03 / PLANNER

The plan names the path and the proof.

Technical choices are tied to acceptance criteria, risks and verification before code changes.

INPUT

**PRD + experience
contract**

WORK

**Scope ·
dependencies ·
failure modes**

OUTPUT

**Build and
verification plan**

STOP

**Unsafe or
untestable path**

A plan is useful when the Builder and Tester can tell what done means.

04 / BUILDER

Build the bounded change — and preserve the agreement.

Implementation and tests remain traceable to the decided behaviour and technical plan.

INPUT

Approved plan

WORK

**Code · tests ·
migration safety**

OUTPUT

**Scoped change +
local evidence**

STOP

**Requirement or
plan contradiction**

The Builder does not silently resolve product ambiguity in code.

Evidence answers a claim — not every possible question.

Verification links observed results to the current change, environment and acceptance criteria.

INPUT

**Change +
acceptance**

WORK

**Exercise happy,
refusal, repeat and
failure paths**

OUTPUT

**Results + gaps +
provenance**

STOP

**Environment cannot
support the claim**

Not run is not passed. Old evidence does not prove the current change.

06 / REVIEWER · HONEST MODEL

Why should we trust this change?

Review brings intent, change, evidence and open concerns into one bounded decision. The NFR X-ray and merge score are aspirational here, not runnable proof.

SUPPORTED

Current evidence supports the scoped claim.

UNASSESSED

No conclusion: missing, unavailable or outside scope.

STALE

The evidence belongs to an older change or baseline.

Leadership needs the basis for confidence — not a fabricated product score.

DECISION BOUNDARIES

Ready to merge is not ready to release.

Related evidence supports different decisions.

MERGE

Can this enter the codebase?

Scoped behaviour · review findings · relevant tests · known limitations

RELEASE

Can users safely receive it?

Integrations · configuration · deployment · rollback · operations

A reviewed change is not a blanket guarantee about the production system.

One finding can improve three different things.

Keep reusable capability, product knowledge and delivery work separate – then review each change.

SKILL

Ask about repeats

A reusable refinement question.

DOMAIN

Record the rule

A human-approved product decision.

PRODUCT

Propose a fix

A separately scoped delivery change.

RECHECK

Next change

Did the recurring issue improve?

The loop closes only when learning changes a later decision or delivery outcome.

CONTROLLED ADOPTION

Scale the method — not uncontrolled access.

Good output, permission to act and worthwhile economics are separate things to establish.

BOUNDED AUTHORITY

Read · propose · record

Permitted repositories and data · approved tools · traceable decisions

EXPLICIT APPROVAL

Higher-impact actions

Credentials · permissions · production data · merge · release · deployment

Permission to inspect a repository does not imply permission to deploy.

START SMALL

Adopt around one bottleneck, then inspect the result.

Use a real feature, real owners and explicit evidence boundaries before expanding.

CHOOSE**One bottleneck**

A recurring, consequential delivery problem.

BOUND**One feature**

Named owners, permissions and approval gates.

OBSERVE**Accepted work**

Spend, human effort, elapsed time, quality and rework.

DECIDE**Expand or stop**

Use observed outcomes, not enthusiasm.

The objective is a better accepted outcome — not the cheapest first answer.

THE NEXT CHANGE

Start with one bottleneck. Improve the next change.

The same request is now agreed, designed, planned, built, tested, reviewed and curated through accountable handoffs.

Intent → delivery → evidence → learning

Every feature can improve the product, the domain understanding and the capability used next time.